Chuleta DOM y Eventos

Desarrollo Web en Entorno Cliente · 2º DAW · Unidad 4 · Boletín DOM (1–4), Boletín Eventos (1–15) y Ejercicio Alumnos

Índice

0. Método para empezar cualquier ejercicio

1. Seleccionar elementos
- Colecciones vivas vs estáticas

2. Leer y modificar elementos

3. Crear, mover y borrar elementos

4. Eventos
- Registrar y quitar
- Tipos de evento
- El objeto event

5. "Quiero…" → cómo se hace

6. Cadenas, números y arrays

7. Expresiones regulares

8. Boletín DOM resuelto
- DOM 1 · Tabla de filas × columnas numerada
- DOM 2 · Estilos según el select (change + setAttribute/removeAttribute)
- DOM 3 · Fondo negro / blanco
- DOM 4 · Tabla 3×3 de colores (delegación)

9. Boletín Eventos resuelto
- Ev 1 · Radio marcado → consola
- Ev 2 y 3 · Opción(es) elegida(s) de un select (simple o múltiple)
- Ev 4 · Dos listas múltiples (el HTML trae onclick="agregarProvincia()" etc.)
- Ev 5 · Añadir / quitar el manejador de un botón
- Ev 6 · Ratón sobre un cuadrado y teclas en un input
- Ev 7 · Botonera numérica con un único manejador
- Ev 8 · Impedir dígitos · Ev 9 · Impedir copiar
- Ev 10 · Qué botón del ratón
- Ev 11 · target, currentTarget y this
- Ev 12 · Cancelar el envío si el input está vacío
- Ev 13 · Validación solo con HTML

10. Ejercicios 14 y 15 explicados
- Ev 14 · Validar el formulario con JavaScript, con listas de vacíos y errores
- Ev 15 · Cookies: guardar, recuperar y eliminar

11. Ejercicio Alumnos (clases Alumno y Nota)
- Usar clases ya hechas
- Eventos y menú (codigo.js)
- Registrar alumno
- Rellenar los desplegables
- Registrar nota
- Detalle de un alumno (evento change)
- Notas medias
- Listado con botón Eliminar (review.js)

12. Errores típicos (revisar antes de entregar)

13. De las diapositivas, aún sin usar

0. Método para empezar cualquier ejercicio

#	Pregunta	Se traduce en…
1	**¿Cuándo** pasa algo?	Un `addEventListener` sobre el elemento que **se pulsa / dispara**
2	**¿Qué datos** necesito?	Leer valores (`.value`, `.checked`, `event.key`…) **dentro** de la función
3	**¿Sobre qué elemento** actúo?	El elemento que **cambia**: seleccionarlo y guardarlo en una variable
4	**¿Qué le hago?**	Las acciones, con `if`/`switch` si dependen de algo

Disparador ≠ objetivo. Lo que se pulsa lleva el evento; lo que cambia es otra cosa. Ej.: se pulsa el botón "Marcar/Desmarcar", cambia el checkbox. Se pulsa una celda, cambia el `body`.
Antes de escribir: esqueleto con comentarios (`// 1. leer… // 2. crear…`) y luego cada paso a una línea.

1. Seleccionar elementos

Método	Devuelve	Ojo
`getElementById("id")`	**Un** elemento o `null`	Sin `#`
`querySelector("selector")`	El **primero** o `null`	`#id`, `.clase`, `input[name=x]`, `form[name=formulario]`
`querySelectorAll("selector")`	**Lista estática** (una foto)	Recorrer con `for...of`
`getElementsByName("valor")`	**Lista viva**	Solo existe en `document`. Entre paréntesis el **valor** del name
`getElementsByClassName("clase")`	**Lista viva**	Sin punto. Uno solo: `[0]`
`getElementsByTagName("p")`	**Lista viva**	La etiqueta: `"input"`, `"td"`…
`document.body` · `document`	El body · la página entera	No hace falta buscarlos

`Elements` en plural o `All` → lista → no tiene `addEventListener`, `.value` ni `appendChild`.
Selectores CSS útiles: espacio = "dentro de" (`"#lstProvincias option"`, `"#capa td"`) · coma = "o" (`"#a option, #b option"`) · `etiqueta[atributo=valor]`.
Mayúsculas: el valor del atributo distingue mayúsculas: `name="Apellidos"` ≠ `input[name=apellidos]`.
No usar atajos tipo `formulario.algo` (el name como variable): usar siempre la ruta completa.

Colecciones vivas vs estáticas

Viva (cambia mientras la recorres)	Estática (foto)
`getElementsBy*`, recorrer un `<select>`, `select.options`, `elem.children`	`querySelectorAll`

Si vas a **mover o borrar** elementos dentro de un bucle, recorre una lista **estática**: con una viva, al sacar uno los demás se desplazan y **te saltas alguno**.

2. Leer y modificar elementos

Quiero…	Cómo
Lo escrito en `input`/`select`/`textarea`	`elem.value` (siempre **texto**). En un select, el value de la opción elegida
El texto de `p`, `td`, `button`, `option`	`elem.textContent` (en option también `.text`)
HTML de dentro / vaciar / saltos de línea	`elem.innerHTML` · `innerHTML = ""` · ` ` · `innerHTML +=` para acumular
¿Radio o checkbox marcado?	`elem.checked` (se lee y se cambia). El `value` es fijo
¿Opción de un select elegida?	`option.selected` · `select.selectedIndex` · `select.options[i]`
Atributos	`setAttribute("style", "color: blue; font-size: 20px")` · `getAttribute("href")` · `removeAttribute("class")` · `hasAttribute("x")`
Un estilo	`elem.style.backgroundColor = "black"` (camelCase: `fontSize`, `fontFamily`)
Clases	`classList.add("x")` · `remove("x")` · `toggle("x")` · `contains("x")` · `className = "x"`
CSS externo	`document.getElementById("estilo").setAttribute("href", "ej08.css")`
Ocultar / mostrar / desactivar	`elem.hidden = true/false` · `elem.disabled = true`
Vaciar un formulario	`formulario.reset()`

value vs textContent: un input tiene **valor**; un `p`/`td`/`div` tiene **texto**. Un `<form>` no tiene value.
Elemento / atributo / valor: `elemento.setAttribute("atributo", "valor")`. `<link>` es el elemento, `href` el atributo.
Valor de style: un único texto CSS: `"color: blue; font-size: 20px; font-family: Helvetica"` (`20px` junto).
Clases sin punto: `classList.add("amarillo")`, `setAttribute("class", "claseEstilo")`. El punto solo en CSS y selectores.
Prioridad: el `style` del propio elemento gana a una clase.
Fuente: `font-size`, `font-family`, `font-weight`, `font-style`. `width`/`height` son la caja.

3. Crear, mover y borrar elementos

Método	Para qué
`document.createElement("td")`	Crea en **memoria**. Nombre sin `< >`. Siempre con `document.`
`padre.appendChild(hijo)`	Mete al final. Solo **elementos**. Si el hijo ya está en la página, lo **mueve**
`padre.append(a, "texto")` · `prepend` · `before` · `after`	Variantes (admiten texto)
`tabla.insertRow()` · `fila.insertCell()`	Atajos de tablas
`elem.remove()` · `elem.cloneNode(true)`	Quitar · copiar (true = con hijos)

Regla: si el elemento **ya existe** en el HTML → se coge y se modifica. Si **lo creas tú** → `createElement` + `appendChild`.
No encadenar: `createElement("tr").appendChild(celda)` devuelve la celda. Crear y meter en dos líneas.
Contenedor: un `div` vacío con id en el HTML es el hueco donde va lo que genera tu JS.
Trampa VS Code: escribir `createElement` sin `document.` mete `require("react")` arriba y rompe todo.

```
// Crear una opción y meterla en un select:  <option value="SE">Sevilla</option>
const opt = document.createElement("option");
opt.value = codigo;           // lo que se envía
opt.textContent = provincia;  // lo que se ve
select.appendChild(opt);
```

4. Eventos

Registrar y quitar

Forma	Notas
`elem.addEventListener("click", miFuncion)`	**No devuelve nada**: nunca `const x = … addEventListener(…)`. `miFuncion` sin paréntesis
`elem.removeEventListener("click", miFuncion)`	Mismo tipo y **misma función con nombre**
`onclick="miFuncion()"` en el HTML	Hace lo mismo: solo hay que definir la función. No usarlo a la vez que addEventListener

miFuncion = pasarla para **más tarde** · **miFuncion()** = ejecutarla **ahora**.
El `addEventListener` puede ir **dentro de otra función**: el evento se pone cuando esa función se ejecuta.
Leer los datos **dentro** de la función del evento, nunca al cargar (los inputs estarían vacíos).

Tipos de evento

Evento	Salta cuando…	Dónde se pone
`click`	Clic con el botón izquierdo	Botón, celda, contenedor (delegación)
`change`	Cambia la opción de un select / el valor de un input	El `<select>`
`submit`	Se va a enviar el formulario (botón submit o Enter)	El **`<form>`**, no el botón
`mouseover` / `mouseout`	El ratón entra / sale (también `mouseenter`/`mouseleave`)	El elemento
`mousedown`	Se pulsa **cualquier** botón del ratón	`document` para toda la página
`contextmenu`	Clic derecho (abre el menú)	`document`
`keydown`	Se pulsa una tecla	El input donde se escribe
`copy`	Se copia (Ctrl+C o clic derecho)	El input

El objeto event

Propiedad / método	Qué es
`event.target`	Dónde se hizo clic de verdad (cambia según dónde pulses)
`event.currentTarget` · `this`	El elemento que **tiene** el manejador (fijo). `this` va solo, no `event.this`
`event.target.tagName`	Etiqueta en MAYÚSCULAS: `"BUTTON"`, `"INPUT"`, `"TD"`, `"P"` (N mayúscula)
`event.type`	Tipo: `"mouseover"`, `"click"`…
`event.clientX` / `clientY`	Coordenadas del ratón en la ventana
`event.key`	Tecla pulsada: `"a"`, `"5"`, `"Enter"`, `"Shift"`
`event.button`	0 izquierdo · 1 rueda · 2 derecho
`event.preventDefault()`	Cancela la acción por defecto (escribir la tecla, copiar, abrir menú, enviar el form)
`event.stopPropagation()`	El evento deja de subir a los padres

Truco: `console.log(event)` dentro de la función y desplegarlo en la Consola para ver todas sus propiedades.
Propagación: el evento sube del hijo a los padres (P → DIV → FORM → BODY → document). Por eso un manejador en un contenedor se entera de los clics de dentro.
Delegación: un solo `addEventListener` en el contenedor + `if (event.target.tagName == "…")`. Sirve también para elementos que **aún no existen** al cargar (creados con JS).

5. "Quiero…" → cómo se hace

Quiero…	Receta	Ejercicio
Generar una tabla	Bucles anidados: `tr` en el de filas, `td` en el de columnas; contador; vaciar contenedor y meter la tabla al final	DOM 1, 4
Aplicar estilos según un select	`change` + `select.value` + `switch`; función `limpiar()` al principio	DOM 2
Alternar entre dos estados	`if/else` mirando el estado actual, o `classList.toggle` + `contains`. Nunca dos `if` seguidos	DOM 3
Saber qué radio está marcado	`for (const r of getElementsByName("x")) if (r.checked) …`	Ev 1
Saber qué opciones de un select están elegidas	`for (const o of select) if (o.selected) …`	Ev 2, 3
Mover opciones entre dos selects	`querySelectorAll("#origen option")` + `if (o.selected) destino.appendChild(o)`	Ev 4
Activar / desactivar un botón	`addEventListener` / `removeEventListener` dentro de funciones	Ev 5
Reaccionar al ratón / teclado	`mouseover`/`mouseout`, `keydown` + propiedades de event	Ev 6

Teclado o botonera con muchos botones	Delegación en el div + `event.target.value` + `pantalla.value +=`	Ev 7
Impedir escribir ciertas teclas	`keydown` + `if ("0123456789".includes(event.key))` `event.preventDefault()`	Ev 8
Impedir copiar / menú contextual	`copy` / `contextmenu` + `preventDefault()`	Ev 9, 10
Que no se envíe un formulario inválido	`submit` en el form + `preventDefault()` si algo falla	Ev 12, 14
Validar formato	HTML: `required pattern="…"` · JS: `/^…$/.test(valor)`	Ev 13, 14
Guardar datos en el navegador	`document.cookie` (ver ejercicio 15)	Ev 15
Usar clases ya hechas	`new Clase(...)` + getters con `()`	Alumnos
Mostrar solo una sección del menú	`ocultarTodo()` + `style.display = "block"` según `event.target.id`	Alumnos
Botón Eliminar en cada fila	`dataset` + `confirm` + `splice` + volver a pintar	Alumnos
Validar antes de guardar	Comprobaciones con `alert` + `return` al principio; lo bueno al final	Ev 4, formulario

6. Cadenas, números y arrays

Elemento	Ejemplo
`cad.length`	Nº de caracteres: `"3".length` → 1 (¡no es el número 3!)
`cad.charAt(0)` · `cad[0]`	Primer carácter (charAt con paréntesis)
`cad.slice(0, 3)`	`"Morales".slice(0,3)` → `"Mor"`
`cad.split(" ")`	`"Morales Pacheco".split(" ")[1]` → `"Pacheco"` · `"5*2".split("*")[1]` → `"2"`
`grande.includes(pequeña)`	`"0123456789".includes("5")` → true. En JS no existe `contains`
`trim()` · `toUpperCase()` · `toLowerCase()`	Quitar espacios de los lados · mayúsculas · minúsculas
`parseInt("12")` · `parseFloat("2.5")`	Texto → número (parseInt corta decimales)
`arr.push(x)` · `arr.join(" ")` · `arr.length`	Añadir · unir en texto · tamaño
`find` · `filter` · `findIndex` · `splice(i, 1)`	Buscar · filtrar · posición · borrar

> `"5" + "55"` → `"555"` (texto + texto pega; convertir antes de sumar). `"5" - "-"` → `NaN`.
> Un solo `=` asigna (el if siempre entra); comparar con `===`/`==`.
> `isNaN(" ")` es false: el espacio "pasa" como número.

```
// Objeto y array de objetos
let alumnos = [];                          // FUERA de las funciones (si no, se vacía en cada clic)
const alumno = { nombre: "Ana", correo: "ana@x.com" };
alumnos.push(alumno);
alumno.nombre;  alumno["correo"];
```

7. Expresiones regulares

Pieza	Significa	Pieza	Significa
`[0-9]`	Un dígito	`+`	Una o más veces
`[a-z]` `[A-Z]`	Minúscula / mayúscula	`*`	Cero o más veces
`[6-9]`	Un carácter del 6 al 9	`( )`	Grupo
`{3}` `{2,4}`	Exactamente 3 / entre 2 y 4	`|`	"o"
`\.`	Punto literal	`.`	Cualquier carácter

Campo	Expresión (atributo `pattern`)	Ejemplo válido
Nombre / apellidos	`[A-ZÁÉÍÓÚÑ][a-záéíóúñ]+( [A-ZÁÉÍÓÚÑ][a-záéíóúñ]+)*`	Pérez López
Fecha DD/MM/AAAA	`(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/[0-9]{4}`	17/01/2023
DNI	`[0-9]{7,8}[A-Z]`	12345678Z
Correo	`[a-z._\-]+@[a-z_\-]+\.[a-z]{2,4}`	correo.prueba@dominio.info
Usuario IDEA	`[a-z]{7}[0-9]{3}`	rmorpac123
Teléfono	`[6-9][0-9]{8}`	612345678
Twitter	`@.{4,15}`	@moha_daw
Código postal	`[0-9]{5}`	41001

HTML (ej. 13)	JavaScript (ej. 14)
`<input required pattern="[0-9]{7,8}[A-Z]">`	`/^[0-9]{7,8}[A-Z]$/.test(valor)`
Comprueba el texto entero solo	Hay que poner `^` y `$`
Guion dentro de `[ ]` escapado: `\-` (si no, Chrome ignora la expresión)	La `/` literal se escapa: `\/`
Valida el navegador al enviar	Validas tú en el `submit`

Equivalencias con Spring: `@NotBlank` → `required` · `@Pattern` → `pattern` · `@Size` → `minlength/maxlength`.

8. Boletín DOM resuelto

DOM 1 · Tabla de filas × columnas numerada

```
document.getElementById("boton").addEventListener("click", crearTabla);

function crearTabla() {
    const filas = parseInt(document.querySelector("input[name=filas]").value);
    const columnas = parseInt(document.querySelector("input[name=columnas]").value);
    const tabla = document.createElement("table");
    let numero = 0;
    for (let i = 0; i < filas; i++) {
        const fila = document.createElement("tr");
        for (let j = 0; j < columnas; j++) {
            const celda = document.createElement("td");
            numero++;
            celda.textContent = numero;
            fila.appendChild(celda);
        }
        tabla.appendChild(fila);                // la fila, cuando está completa
    }
    const contenedor = document.getElementById("tabla");
    contenedor.innerHTML = "";                  // no se acumulan tablas
    contenedor.appendChild(tabla);              // una vez, al final
}
```

DOM 2 · Estilos según el select (change + setAttribute/removeAttribute)

```
document.querySelector("select[name=opciones]").addEventListener("change", opcionDesplegable);
const texto = document.getElementById("texto");    // elementos: pueden ir fuera
const enlace = document.getElementById("estilo");  // el <link>

function limpiar() {
    texto.removeAttribute("style");
    texto.removeAttribute("class");
    enlace.removeAttribute("href");
}

function opcionDesplegable() {
    const opcion = document.querySelector("select[name=opciones]").value;  // valor: dentro
    limpiar();
    switch (opcion) {
        case "atributoStyle":   texto.setAttribute("style", "color: blue; font-size: 20px;
font-family: Helvetica"); break;
        case "asignandoClases": texto.setAttribute("class", "claseEstilo"); break;
        case "estilosPagina":   enlace.setAttribute("href", "ej08.css"); break;
        // "quitarEstilos": ya lo hace limpiar()
    }
}
```

Variables con `const`/`let` solo existen en su función: para usarlas en otra, pasarlas como parámetro o declararlas fuera. Fuera se pueden declarar **elementos**; los **valores** se leen dentro. Un parámetro tapa a una variable de fuera con el mismo nombre.

DOM 3 · Fondo negro / blanco

```
// Versión con style (estado inicial fijado al cargar)
document.getElementById("boton").addEventListener("click", fondos);
document.body.style.background = "white";

function fondos() {
    const boton = document.getElementById("boton");
    if (document.body.style.background == "white") {
        document.body.style.background = "black";
        boton.textContent = "Fondo Blanco";
    } else {                                        // else, NO dos if seguidos
        document.body.style.background = "white";
        boton.textContent = "Fondo Negro";
    }
}

// Versión con clase (.oscuro { background-color: black; } en el CSS)
function fondos() {
    const boton = document.getElementById("boton");
    document.body.classList.toggle("oscuro");
    boton.textContent = document.body.classList.contains("oscuro") ? "Fondo Blanco" : "Fondo
Negro";
}
```

DOM 4 · Tabla 3×3 de colores (delegación)

```
document.getElementById("boton").addEventListener("click", colorCelda);
document.getElementById("capa").addEventListener("click", cambiarFondo);   // capa ya existe

function colorCelda() {
    const colores = ["#ff0000", "#00ff00", "#0000ff", "#ffff00", "#ff00ff",
                     "#00ffff", "#ffa500", "#808080", "#000000"];
    let contador = 0;
    const capa = document.getElementById("capa");
    const tabla = document.createElement("table");
    capa.innerHTML = "";
    for (let i = 0; i < 3; i++) {
        const fila = document.createElement("tr");
        tabla.appendChild(fila);
        for (let j = 0; j < 3; j++) {
            const celda = document.createElement("td");
            celda.style.backgroundColor = colores[contador];
            contador++;
            fila.appendChild(celda);
        }
    }
    capa.appendChild(tabla);
}

function cambiarFondo(event) {
    if (event.target.tagName == "TD") {
        document.body.style.backgroundColor = event.target.style.backgroundColor;
    }
}
```

En el CSS: `td { width: 50px; height: 50px; }` (las reglas se aplican también a lo creado con JS).

9. Boletín Eventos resuelto

Ev 1 · Radio marcado → consola

```
document.getElementById("consultar").addEventListener("click", consultarDatos);
function consultarDatos() {
    for (const input of document.getElementsByName("actores")) {
        if (input.checked) console.log(input.value);
    }
}
```

Ev 2 y 3 · Opción(es) elegida(s) de un select (simple o múltiple)

```
document.querySelector("input[name=boton]").addEventListener("click", mostrarInfo);
function mostrarInfo() {
    const select = document.querySelector("select[name=provincias]");
    for (const provincia of select) {
        if (provincia.selected) console.log(provincia.textContent + " - " + provincia.value);
    }
}
// Sin bucle (solo la primera): select.options[select.selectedIndex]
```

Ev 4 · Dos listas múltiples (el HTML trae onclick="agregarProvincia()" etc.)

```
function agregarProvincia() {
    const codigo = document.getElementById("txtCodigo").value.trim();
    const provincia = document.getElementById("txtProvincia").value.trim();
    if (codigo === "" || provincia === "") { alert("Rellena los dos campos"); return; }
    for (const o of document.querySelectorAll("#lstProvincias option, #lstDestino option")) {
        if (o.value === codigo) { alert("La provincia ya existe"); return; }
    }
    const opt = document.createElement("option");
    opt.value = codigo;
    opt.textContent = provincia;
    document.getElementById("lstProvincias").appendChild(opt);
}

function pasarDerecha() {
    const destino = document.getElementById("lstDestino");
    for (const o of document.querySelectorAll("#lstProvincias option")) {   // foto estática
        if (o.selected) destino.appendChild(o);                              // appendChild MUEVE
    }
}

function pasarIzquierda() {
    const lista = document.getElementById("lstProvincias");
    for (const o of document.querySelectorAll("#lstDestino option")) {
        if (o.selected) lista.appendChild(o);
    }
}
```

Ev 5 · Añadir / quitar el manejador de un botón

```
const marcar = document.getElementById("verano");          // el checkbox (lo que cambia)
function marcarVerano() { marcar.checked = !marcar.checked; }
function addManejador() {
    document.getElementById("botonMarcar").addEventListener("click", marcarVerano);
}
function deleteManejador() {
    document.getElementById("botonMarcar").removeEventListener("click", marcarVerano);
}
```

Ev 6 · Ratón sobre un cuadrado y teclas en un input

```
const cuadrado = document.getElementById("cuadrado");
cuadrado.addEventListener("mouseover", pintar);
cuadrado.addEventListener("mouseout", despintar);
document.getElementById("txtEntrada").addEventListener("keydown", tecla);

function pintar(event) {
    cuadrado.classList.add("amarillo");
    document.getElementById("salida").textContent = "Evento: " + event.type + " - Objeto: " +
        event.target.id + " - X: " + event.clientX + " - Y: " + event.clientY;
}
function despintar() {
    cuadrado.classList.remove("amarillo");
    document.getElementById("salida").textContent = "";
}
function tecla(event) {
    document.getElementById("salida").innerHTML += "Tecla pulsada: " + event.key + "<br>";
}
```

Ev 7 · Botonera numérica con un único manejador

```
document.getElementById("teclado").addEventListener("click", tecla);
function tecla(event) {
    if (event.target.tagName == "INPUT") {            // <input type="button"> → "INPUT"
        document.getElementById("salida").value += event.target.value;
    }
}
```

Ev 8 · Impedir dígitos · Ev 9 · Impedir copiar

```
document.getElementById("txtEntrada").addEventListener("keydown", function (event) {
    if ("0123456789".includes(event.key)) event.preventDefault();
});
document.getElementById("txtEntrada").addEventListener("copy", function (event) {
    event.preventDefault();
});
```

Ev 10 · Qué botón del ratón

```
document.addEventListener("mousedown", mostrarBoton);
document.addEventListener("contextmenu", function (event) { event.preventDefault(); });
function mostrarBoton(event) {
    const salida = document.getElementById("salida");
    if (event.button == 0) salida.textContent = "Has pulsado el botón izquierdo";
    else if (event.button == 2) salida.textContent = "Has pulsado el botón derecho";
}
```

Ev 11 · target, currentTarget y this

```
document.getElementById("formulario").addEventListener("click", function (event) {
    console.log(event.target.tagName);         // P, DIV o FORM: donde se hizo clic
    console.log(event.currentTarget.tagName);  // FORM siempre: quien tiene el manejador
    console.log(this);                         // <form> siempre
});
```

Ev 12 · Cancelar el envío si el input está vacío

```
document.querySelector("form[name=formulario]").addEventListener("submit", enviarForm);
function enviarForm(event) {
    const texto = document.querySelector("input[name=txtTexto]").value.trim();
    if (texto === "") {
        alert("No hay texto, no se puede enviar");
        event.preventDefault();
    }
}
```

Ev 13 · Validación solo con HTML

Añadir a cada input `required` y `pattern="…"` con las expresiones del apartado 7. Ejemplo: `<input type="text" name="telefono" required pattern="[6-9][0-9]{8}" />`. Sin JavaScript.

10. Ejercicios 14 y 15 explicados

Ev 14 · Validar el formulario con JavaScript, con listas de vacíos y errores

Enunciado: validar el formulario del ejercicio 13 desde JS. En la capa de salida se muestran una lista con los campos **vacíos** y otra con los campos **con errores**. Si todo es correcto, el formulario se envía.

```
document.querySelector("form[name=formulario]").addEventListener("submit", validarFormulario);

// Un objeto por campo: su name, el texto que se muestra y la expresión que debe cumplir
const campos = [
    { nombre: "apellidos",       texto: "Apellidos",           regex: /^[A-ZÁÉÍÓÚÑ][a-záéíóúñ]+
( [A-ZÁÉÍÓÚÑ][a-záéíóúñ]+)*$/ },
    { nombre: "nombre",          texto: "Nombre",              regex: /^[A-ZÁÉÍÓÚÑ][a-záéíóúñ]+
( [A-ZÁÉÍÓÚÑ][a-záéíóúñ]+)*$/ },
    { nombre: "fechaNacimiento", texto: "Fecha de nacimiento", regex: /^(0[1-9]|[12][0-
9]|3[01])\/(0[1-9]|1[0-2])\/[0-9]{4}$/ },
    { nombre: "dni",             texto: "DNI",                 regex: /^[0-9]{7,8}[A-Z]$/ },
    { nombre: "email",           texto: "Correo electrónico",  regex: /^[a-z._-]+@[a-z_-]+\.[a-
z]{2,4}$/ },
    { nombre: "usuario",         texto: "Usuario IDEA",        regex: /^[a-z]{7}[0-9]{3}$/ },
    { nombre: "telefono",        texto: "Teléfono",            regex: /^[6-9][0-9]{8}$/ },
    { nombre: "twitter",         texto: "Usuario de Twitter",  regex: /^@.{4,15}$/ }
];

function validarFormulario(event) {
    const vacios = [];
    const errores = [];

    for (const campo of campos) {
        const valor = document.querySelector("input[name=" + campo.nombre + "]").value.trim();
        if (valor === "") {
            vacios.push(campo.texto);
        } else if (!campo.regex.test(valor)) {
            errores.push(campo.texto);
        }
    }

    const salida = document.getElementById("salida");
    salida.innerHTML = "";

    if (vacios.length === 0 && errores.length === 0) {
        return;                                  // todo bien: no se cancela y se envía
    }

    event.preventDefault();                      // algo falla: no se envía
    if (vacios.length > 0)  salida.appendChild(crearLista("Campos vacíos:", vacios));
    if (errores.length > 0) salida.appendChild(crearLista("Campos con errores:", errores));
}

// Devuelve un div con un título y una lista <ul> con un <li> por elemento
function crearLista(titulo, elementos) {
    const bloque = document.createElement("div");
    const p = document.createElement("p");
    p.textContent = titulo;
    bloque.appendChild(p);
    const ul = document.createElement("ul");
    for (const elemento of elementos) {
        const li = document.createElement("li");
        li.textContent = elemento;
        ul.appendChild(li);
    }
    bloque.appendChild(ul);
    return bloque;
}
```

Cómo funciona:

1. **Array de objetos `campos`:** en vez de 8 bloques `if` casi iguales, cada campo se describe con su `name`, su texto y su expresión. Un solo bucle los valida todos; para añadir un campo basta con una línea más.

2. **Evento `submit` en el formulario** (como el ej. 12): salta con el botón y con Enter, antes de enviar.
3. **El bucle:** construye el selector con el name (`"input[name=" + campo.nombre + "]"`), lee el valor con `trim()` y lo clasifica: si está vacío va a `vacios`; si no está vacío pero `regex.test(valor)` da `false`, va a `errores`. El `else if` evita que un campo vacío cuente también como error.
4. **Decisión:** si los dos arrays están vacíos, `return` y el navegador envía el formulario. Si no, `preventDefault()` y se pintan las listas.
5. **`crearLista`:** función auxiliar reutilizable; mismo patrón que la tabla (crear, rellenar, `appendChild`), pero con `ul`/`li`. Se vacía la salida al principio para no acumular mensajes de intentos anteriores.
6. **Expresiones en JS:** entre `/ /`, con `^` (principio) y `$` (final) para que se compruebe el texto entero, y la `/` de la fecha escapada como `\/`.

Ev 15 · Cookies: guardar, recuperar y eliminar

Enunciado: tres botones. **Guardar** crea la cookie clave=valor con lo escrito; **Recuperar** muestra en un `alert` el valor de la cookie cuya clave se ha escrito; **Eliminar** borra la cookie de esa clave. Después de cada acción se resetea el formulario.

```
document.querySelector("input[name=btnGuardar]").addEventListener("click", guardar);
document.querySelector("input[name=btnRecuperar]").addEventListener("click", recuperar);
document.querySelector("input[name=btnEliminar]").addEventListener("click", eliminar);
const formulario = document.querySelector("form[name=formulario]");

function guardar() {
    const clave = document.querySelector("input[name=txtClave]").value.trim();
    const valor = document.querySelector("input[name=txtValor]").value.trim();
    if (clave === "") { alert("Debes escribir una clave"); return; }
    document.cookie = encodeURIComponent(clave) + "=" + encodeURIComponent(valor) +
                      "; max-age=" + 60 * 60 * 24 * 7 + "; path=/";    // 7 días
    formulario.reset();
}

function recuperar() {
    const clave = document.querySelector("input[name=txtClave]").value.trim();
    if (clave === "") { alert("Debes escribir una clave"); return; }
    const valor = leerCookie(clave);
    if (valor === null) alert("No existe ninguna cookie con la clave " + clave);
    else alert("El valor de la cookie " + clave + " es: " + valor);
    formulario.reset();
}

function eliminar() {
    const clave = document.querySelector("input[name=txtClave]").value.trim();
    if (clave === "") { alert("Debes escribir una clave"); return; }
    if (leerCookie(clave) === null) {
        alert("No existe ninguna cookie con la clave " + clave);
    } else {
        document.cookie = encodeURIComponent(clave) + "=; max-age=0; path=/";   // caduca ya
        alert("Cookie " + clave + " eliminada");
    }
    formulario.reset();
}

// document.cookie = "clave1=valor1; clave2=valor2" → partir y buscar
function leerCookie(clave) {
    for (const cookie of document.cookie.split("; ")) {
        const partes = cookie.split("=");
        if (decodeURIComponent(partes[0]) === clave) return decodeURIComponent(partes[1]);
    }
    return null;
}
```

Cómo funciona `document.cookie`:

Acción	Explicación
Guardar	`document.cookie = "clave=valor; max-age=SEGUNDOS; path=/"`. Aunque parezca que machaca, cada asignación **añade o sobrescribe una sola** cookie y deja las demás. `max-age` en segundos; sin él, la cookie desaparece al cerrar el navegador. `path=/` la hace visible en todo el sitio.
Leer	Al leer `document.cookie` se obtienen **todas** juntas en un texto: `"color=azul; idioma=es"`. Hay que partir por `"; "` y cada trozo por `"="`. Por eso existe `leerCookie`, que devuelve el valor o `null`.
Eliminar	No hay método para borrar: se guarda la **misma clave** con `max-age=0` (o `expires` en una fecha pasada) y caduca al momento.
encode / decode	`encodeURIComponent` convierte espacios, `;` y `=` para que no rompan el formato; `decodeURIComponent` lo deshace al leer.
reset()	Método del formulario que vacía todos sus campos de golpe.

⚠️ **Probar con Live Server** (`http://127.0.0.1…`): abriendo el HTML con doble clic (`file://`) Chrome no guarda cookies y parece que no funciona.

11. Ejercicio Alumnos (clases Alumno y Nota)

Qué es: una aplicación con menú (Registro, Registrar Notas, Listado, Notas Medias, Cerrar Sesión) que usa las clases `Alumno` y `Nota` que da el profe en `clases.js`. El código va en `codigo.js` y el listado en `review.js`.

Usar clases ya hechas

Qué	Cómo
Crear un objeto	`new Alumno(nombre, apellidos, usuario, contrasena, correo)` · `new Nota(asignatura, nota)` — en el orden del constructor
Leer datos privados (`#nombre`)	Solo con getters **y con paréntesis**: `alumno.getNombre()`. `alumno.nombre` da `undefined`
Métodos de `Alumno`	`getNombre()` · `getApellidos()` · `getUsuario()` · `getContrasena()` · `getCorreo()` · `getLongitudArrayNotas()` · `getNota(i)` · `getAsignaturaYNota(i)` → `"cliente:7.5"` · `altaNota(oNota)` → `true` si **ya existía** (y no la guarda)
Métodos de `Nota`	`getAsignatura()` · `getNota()` · `setAsignatura(v)` · `setNota(v)`
Ver un objeto en consola	`console.log(alumno)` sale como `Alumno {}` (los privados no se ven). Comprobar con `alumnos[0].getNombre()`

Un objeto `Alumno` no es un elemento HTML: no se puede meter con `appendChild`; hay que crear `<option>`, `<td>`… con sus datos.
Una misma `<option>` no puede estar en dos selects: crear una para cada uno.
`reset()` solo existe en `<form>`, no en selects ni botones.
Ids parecidos: `selAlumnoDetalle` es el select; `detalleAlumno` es el div de la tabla.
Varios scripts: se cargan en orden (`clases.js` → `codigo.js` → `review.js`) y comparten lo declarado fuera de las funciones.

VS Code no sugiere los métodos de `alumno.` ni de `event.` porque no sabe su tipo (el array empieza vacío, la clase está en otro archivo). El icono `abc` son solo palabras del archivo. **El código**

funciona igual escribiéndolo a mano. Para que sugiera: `/** @type {Alumno[]} */` encima de `let alumnos = [];` · `/** @param {Event} event */` (o `MouseEvent`, `KeyboardEvent`) encima de la función · `/** @param {string} cadena */` para un texto.

Eventos y menú (codigo.js)

```
document.getElementById("btnCrearUsuario").addEventListener("click", crearUsuario);
document.getElementById("btnRegistrarAlumno").addEventListener("click", registrarAlumno);
document.getElementById("registroAlumnos").addEventListener("click", mostrarRegistro);
document.getElementById("registroNotas").addEventListener("click", mostrarRegistro);
document.getElementById("listadoAlumnos").addEventListener("click", mostrarRegistro);
document.getElementById("notasMedias").addEventListener("click", mostrarRegistro);
document.getElementById("cerrarSesion").addEventListener("click", mostrarRegistro);
document.getElementById("btnRegistrarNotas").addEventListener("click", registrarNota);
document.getElementById("selAlumnoDetalle").addEventListener("change", mostrarDetalle);

ocultarTodo();
```

```
function ocultarTodo(){
    document.getElementById("formRegistrarAlumno").style.display = "none";
    document.getElementById("formRegistrarNotas").style.display = "none";
    document.getElementById("formDetalle").style.display = "none";
    document.getElementById("lNotasMedias").style.display = "none";
    document.getElementById("detalleAlumno").style.display = "none";
    document.getElementById("listado").style.display = "none";
}
```

```
/** @param {Event} event */
function mostrarRegistro(event){
    event.preventDefault();
    ocultarTodo();
    switch (event.target.id) {
        case "registroAlumnos":
            document.getElementById("formRegistrarAlumno").style.display="block";
            break;

        case "registroNotas":
            document.getElementById("formRegistrarNotas").style.display="block";
            break;

        case "listadoAlumnos":
            document.getElementById("listado").style.display="block";
            document.getElementById("formDetalle").style.display="block";
            document.getElementById("selAlumnoDetalle").value="";
            mostrarListado();
            break;

        case "notasMedias":
            document.getElementById("lNotasMedias").style.display="block";
            mostrarMedias();
            break;

        default:
            break;
    }
}
```

Menú: un solo manejador para todos los enlaces. Primero `ocultarTodo()` y luego `style.display = "block"` solo a la sección del enlace pulsado (`event.target.id`). `preventDefault()` evita que el `href="#"` haga nada. Se llama a `ocultarTodo()` también al cargar.

Registrar alumno

```
function registrarAlumno(){
    const nombre= document.querySelector("input[name=nombre]").value;
    const apellidos= document.querySelector("input[name=Apellidos]").value;
    const usuario= document.querySelector("input[name=usuario]").value;
    const contrasena= document.querySelector("input[name=contrasena]").value;
    const correo= document.querySelector("input[name=correo]").value;

    if(nombre=="" || apellidos== "") {
        alert("El nombre y los apellidos son obligatorios");
        return;
    }

    if(usuario==""){
        alert("El usuario no debe estar vacio");
        return;
    }

    if(validarContrasena(contrasena) == false){
        alert("Debe contener un numero, letra minuscula, letra mayuscula y tener una longitud
minima de 6.");
        return;
    }
    if( validarCorreo(correo) == false){
        alert("Debe contener un @, y un .");
        return;
    }

    const alumno= new Alumno(nombre, apellidos, usuario, contrasena, correo);
    alumnos.push(alumno);
    actualizarDesplegables();
    alert("Alumno registrado");
    document.getElementById("formRegistrarAlumno").reset();
}
```

Orden: leer cada input en su variable → validar con esas variables (cada una con `alert + return`) → `new Alumno(...)` → `push` → refrescar desplegables → aviso → `reset()`. `validarContrasena`, `validarCorreo` y `crearUsuario` son las del Ejercicio Formularios:

```
function crearUsuario() {
    const nombre=document.querySelector("input[name=nombre]").value;
    const apellido=document.querySelector("input[name=Apellidos]").value;

    let primerApellido=apellido.split(" ")[0];
    let segundoApellido=apellido.split(" ")[1];

    let nombreUsuario=nombre[0];

    for (let i = 0; i < primerApellido.length ; i++) {
        let letra = primerApellido[i];
        if(i<=2){
            nombreUsuario+=letra;
        }
    }

    for (let i = 0; i < segundoApellido.length ; i++) {
        let letra = segundoApellido[i];
        if(i<=2){
            nombreUsuario+=letra;
        }
    }
    nombreUsuario=nombreUsuario.toLowerCase();
    document.querySelector("input[name=usuario]").value=nombreUsuario;
}
```

```
function validarContrasena(contrasena) {
    let contieneNumero=false;
    const abecedarioMin='abcdefghijklmnopqrstuvwxyz';
    const abecedarioMayusculas = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';
    const numerosCadena = '0123456789';
    let longitudCorrecta=false;
    let letraMin=false;
    let letraMay=false;
    let contrasenaValida=false;

    if(contrasena.length>=6) {
        longitudCorrecta=true;
    }
    for (let i = 0; i < contrasena.length; i++) {
        let letra  = contrasena[i];
        if(numerosCadena.includes(letra)){
            contieneNumero=true;
        }
        if(abecedarioMin.includes(letra)){
            letraMin=true;
        }
        if(abecedarioMayusculas.includes(letra)){
            letraMay=true;
        }
    }
    if(longitudCorrecta==true && contieneNumero==true && letraMin==true && letraMay==true) {
        contrasenaValida=true;
    }
    return contrasenaValida;
}
```

```
function validarCorreo(correo){
    let correoValido=false;

    if(correo.includes("@") && correo.includes(".")){
        correoValido=true;
    }
    return correoValido;
}
```

Rellenar los desplegables

```
function actualizarDesplegables(){
    const select=document.getElementById("selAlumno");
    select.innerHTML="";
    const detalle=document.getElementById("selAlumnoDetalle");
    detalle.innerHTML="";

    // Primera opción vacía: así el evento change salta al elegir CUALQUIER alumno
    const opcionVacia=document.createElement("option");
    opcionVacia.value="";
    opcionVacia.textContent="Selecciona un alumno";
    detalle.appendChild(opcionVacia);

    for (const alumno of alumnos) {
        const option = document.createElement("option");
        option.value=alumno.getUsuario();
        option.innerHTML=alumno.getNombre() + " " + alumno.getApellidos();
        select.appendChild(option);

        const option2=document.createElement("option");
        option2.value=alumno.getUsuario();
        option2.innerHTML=alumno.getNombre() + " " + alumno.getApellidos();
        detalle.appendChild(option2);
    }
}
```

Vaciar los selects y volver a crear todas las opciones desde el array. La opción vacía "Selecciona un alumno" evita la **trampa del `change`**: si el alumno ya está seleccionado (el único o el primero), elegirlo no dispara el evento.

Registrar nota

```
function registrarNota(){
    const usuario=document.getElementById("selAlumno").value;
    const asignatura=document.querySelector("select[name=selNota]").value;
    const nota=parseFloat(document.querySelector("input[name=nota]").value);

    if(isNaN(nota)){
        alert("No puede estar vacio");
        return;
    }
    if(nota<0 || nota>10) {
        alert("La nota debe estar entre 0 y 10");
        return;
    }
    for (const alumno of alumnos) {
        if(usuario==alumno.getUsuario()){
            const obNota= new Nota(asignatura,nota);
            const existe= alumno.altaNota(obNota);
            if(existe){
                alert("Esa asignatura ya tenia la nota guardada");
            } else {
                alert("Nota registrada");
            }
        }
    }

    document.getElementById("formRegistrarNotas").reset();
}
```

> Validar primero que haya número (`isNaN`) y luego el rango. Buscar al alumno con un bucle + `if (usuario == alumno.getUsuario())`. `altaNota` ya guarda la nota: no se hace `push`; su resultado solo sirve para avisar.

Detalle de un alumno (evento change)

```
function mostrarDetalle(){
    const usuario=document.getElementById("selAlumnoDetalle").value;
    const capa = document.getElementById("detalleAlumno");
    capa.innerHTML="";

    // Si se vuelve a "Selecciona un alumno", se oculta el detalle
    if(usuario==""){
        capa.style.display="none";
        return;
    }

    for (const alumno of alumnos) {
        if(usuario==alumno.getUsuario()){
            capa.style.display="block";

            if(alumno.getLongitudArrayNotas()==0){
                capa.textContent="Este alumno no tiene notas registradas";
                return;
            }

            const detalle=document.createElement("table");
            detalle.className="table table-striped";

            for (let i = 0; i < alumno.getLongitudArrayNotas(); i++) {
                let notaAsignatura = alumno.getAsignaturaYNota(i);
                let partirNotaAsignatura=notaAsignatura.split(":");
                let asignatura=partirNotaAsignatura[0];
                let nota=partirNotaAsignatura[1];
                const fila=document.createElement("tr");
                const columnaAsig=document.createElement("td");
                const columnaNota=document.createElement("td");
                columnaAsig.textContent=asignatura;
                columnaNota.textContent=nota;
                fila.appendChild(columnaAsig);
                fila.appendChild(columnaNota);
                detalle.appendChild(fila);
            }

            capa.appendChild(detalle);
        }
    }
}
```

`capa` es el div que ya existe (mostrar con `"block"`, sin `;` dentro de las comillas, y vaciar); `detalle` es la tabla nueva. `getAsignaturaYNota(i).split(":")` separa asignatura y nota.

Notas medias

```
function mostrarMedias(){
    const notas=document.getElementById("lNotasMedias");
    notas.innerHTML="";

    if(alumnos.length==0){
        notas.textContent="No hay alumnos registrados";
        return;
    }

    const tableNotaMedia=document.createElement("table");
    tableNotaMedia.className="table table-striped";

    for (const alumno of alumnos) {
        // 1. Para CADA alumno: la suma empieza en 0
        let sumaNotas=0;
        const numNotas=alumno.getLongitudArrayNotas();

        // 2. Bucle de notas: SOLO sumar
        for (let i = 0; i < numNotas; i++) {
            sumaNotas += alumno.getNota(i);
        }

        // 3. Ya fuera del bucle de notas: la media de este alumno
        let media;
        if(numNotas==0){
            media="Sin notas";
        } else {
            media=(sumaNotas/numNotas).toFixed(2);
        }

        // 4. UNA fila por alumno
        const fila=document.createElement("tr");
        const columnaNombre=document.createElement("td");
        const columnaMedia=document.createElement("td");
        columnaNombre.textContent=alumno.getNombre() + " " + alumno.getApellidos();
        columnaMedia.textContent=media;
        fila.appendChild(columnaNombre);
        fila.appendChild(columnaMedia);
        tableNotaMedia.appendChild(fila);
    }

    notas.appendChild(tableNotaMedia);
}
```

> **Qué va en cada bucle:** el de **alumnos** pone la suma a 0, calcula la media y crea **una** fila; el de **notas** solo suma. Si la fila se crea en el de dentro sale una por nota (y fuera no existe). `toFixed(2)` deja dos decimales; si no hay notas, "Sin notas" (evita dividir entre 0).

Listado con botón Eliminar (review.js)

En el HTML se añade un contenedor: `<div id="listado" style="display: none; width: 1000px; margin: 30px auto"></div>`, que también se oculta en `ocultarTodo()` y se muestra en el `case "listadoAlumnos"` llamando a `mostrarListado()`.

```
// Tabla Bootstrap con nombre, apellidos, usuario y correo, y un botón Eliminar por fila
function mostrarListado(){
    const capa=document.getElementById("listado");
    capa.innerHTML="";

    if(alumnos.length==0){
        capa.textContent="No hay alumnos registrados";
        return;
    }

    const tabla=document.createElement("table");
    tabla.className="table table-striped";

    // Fila de cabecera con <th>
    const cabecera=document.createElement("tr");
    const titulos=["Nombre", "Apellidos", "Usuario", "Correo", ""];
    for (const titulo of titulos) {
        const th=document.createElement("th");
        th.textContent=titulo;
        cabecera.appendChild(th);
    }
    tabla.appendChild(cabecera);

    // Una fila por alumno
    for (const alumno of alumnos) {
        const fila=document.createElement("tr");

        const datos=[alumno.getNombre(), alumno.getApellidos(), alumno.getUsuario(),
alumno.getCorreo()];
        for (const dato of datos) {
            const td=document.createElement("td");
            td.textContent=dato;
            fila.appendChild(td);
        }

        // Celda con el botón Eliminar: guarda el usuario en un atributo data-usuario
        const tdBoton=document.createElement("td");
        const boton=document.createElement("button");
        boton.type="button";
        boton.className="btn btn-danger btn-sm";
        boton.textContent="Eliminar";
        boton.dataset.usuario=alumno.getUsuario();
        boton.addEventListener("click", eliminarAlumno);
        tdBoton.appendChild(boton);
        fila.appendChild(tdBoton);

        tabla.appendChild(fila);
    }

    capa.appendChild(tabla);
}

/** @param {Event} event */
function eliminarAlumno(event){
    const usuario=event.target.dataset.usuario;

    // Buscar la posición del alumno en el array
    let posicion=-1;
    for (let i = 0; i < alumnos.length; i++) {
        if(alumnos[i].getUsuario()==usuario){
            posicion=i;
        }
    }
    if(posicion==-1){
```

```
        return;
    }

    const alumno=alumnos[posicion];
    const nombreCompleto=alumno.getNombre() + " " + alumno.getApellidos();

    // confirm devuelve true si se pulsa Aceptar y false si se pulsa Cancelar
    if(!confirm("¿Está seguro de que desea eliminar al alumno " + nombreCompleto + "?")){
        return;
    }

    alumnos.splice(posicion, 1);          // quitarlo del array
    actualizarDesplegables();             // que desaparezca también de los selects
    document.getElementById("detalleAlumno").style.display="none";
    mostrarListado();                     // volver a pintar la tabla sin él
```

13. De las diapositivas, aún sin usar

Diap.	Contenido
9–10	Recorrer el DOM: `childNodes`, `firstChild`, `lastChild`, `parentNode`, hermanos
17	`outerHTML`: el HTML del elemento incluido él mismo
22–23	`prepend`, `before`, `after`, `replaceWith` · `insertAdjacentHTML("beforeend", html)`
26	`style.cssText` · `style.removeProperty("background")`
29	`elem.onclick = funcion` (solo admite uno; por eso mejor `addEventListener`)
36	Script al final del `body` (o en el head con `defer`) para que el HTML exista al ejecutarse